CSCE883 Machine Learning **Name:** *Taylor Belcher*
Homework 3
10/11/2026

---

1. *(Question 1.)* Gradient propagation and CNN dimensions.

   A. *Gradient propagation.* We have a linear network with L layers with the initial input $h_0 = x$ and at each subsesquent layer l (where $1 \leq l \leq L$), $h_l = \sigma(w_l h_{l-1})$ and $\sigma(z) = \dfrac{1}{1+e^{-z}}$. $|w_l| \leq 1$ and there are no biases.

   (i) We want to estimate an upper bound on $\left|\dfrac{\partial h_L}{\partial x}\right|$ in terms of L. We have that

   $$\begin{aligned} h_L &= \sigma(w_L h_{L-1}) \\ &= \sigma\big(w_L \sigma\big(w_{L-1}\sigma(\cdots\sigma(w_1 x))\big)\big). \end{aligned}$$

   At any given level $l \geq 2$, let $g_l(x)$ denote the remaining nested expression, including the weight w_l, all the way down to the input x:

   $$g_l(x) = w_l \sigma\big(w_{l-1}\sigma(\cdots\sigma(w_1 x))\big), \qquad g_1(x) = w_1 x.$$

   Thus $h_l = \sigma(g_l(x))$. By the chain rule, the partial derivative of h_L with respect to x can be written as

   $$\begin{aligned} \left|\frac{\partial h_L}{\partial x}\right| &= |\sigma'(g_L(x))\, g_L'(x)| \\ &= |\sigma'(g_L(x))|\ \left|w_L\, \sigma'(g_{L-1}(x))\, g_{L-1}'(x)\right| \\ &= |\sigma'(g_L(x))|\ |w_L\, \sigma'(g_{L-1}(x))| \\ &\quad \cdot |w_{L-1}\, \sigma'(g_{L-2}(x))| \cdots |w_2\, \sigma'(g_1(x))|\ |w_1| \\ &= \left(\prod_{l=1}^{L} |w_l|\right)\left(\prod_{l=1}^{L} |\sigma'(g_l(x))|\right) \\ &\leq 1 \cdot \prod_{l=1}^{L} |\sigma'(g_l(x))|\,. \end{aligned}$$

   The inequality follows from $|w_l| \leq 1$ for every layer, so $\prod_{l=1}^{L} |w_l| \leq 1$. Now the derivative of the sigmoid activation function is

   $$\sigma'(z) = -(1+e^{-z})^{-2}(-e^{-z}) = \frac{e^{-z}}{(1+e^{-z})^2}.$$

   To bound this derivative, observe that

   $$1 - \sigma(z) = 1 - \frac{1}{1+e^{-z}} = \frac{e^{-z}}{1+e^{-z}}.$$

Thus we can rewrite the derivative as

$$\sigma'(z) = \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} = \sigma(z)\big(1-\sigma(z)\big).$$

Since $0 < \sigma(z) < 1$, this derivative is positive. Completing the square gives

$$|\sigma'(z)| = \sigma(z)\big(1-\sigma(z)\big) = \frac{1}{4} - \left(\sigma(z) - \frac{1}{2}\right)^2 \leq \frac{1}{4}.$$

Applying this bound to each of the L factors in our product yields

$$\begin{aligned}\left|\frac{\partial h_L}{\partial x}\right| &\leq \prod_{l=1}^{L} |\sigma'(g_l(x))| \\ &\leq \prod_{l=1}^{L} \frac{1}{4} \\ &= \boxed{\left(\frac{1}{4}\right)^L = 4^{-L}}.\end{aligned}$$

Therefore the magnitude of the input derivative is bounded above by 4^{-L}, which decreases exponentially with the number of layers. Increasing the number of layers will decrease the gradient exponentially, making it so that earlier layers will take much longer to train.

(ii) If the weights were not bounded above by 1, then we could counter-balance the derivative of the sigmoid function with a larger product of weights, but this is not a guarantee as the weights would also increase the input to the sigmoid functions and the larger the input, the flatter the curve on the sigmoid gets, which means the derivative would also be getting smaller as it moves into the saturation region. So increasing the weights can help with a vanishing gradient, but it is not a guaranteed solution. More precisely, the weight bound matters because each layer contributes both a weight and a sigmoid derivative:

$$\left|\frac{\partial h_l}{\partial h_{l-1}}\right| = |w_l \sigma'(w_l h_{l-1})| \leq \frac{|w_l|}{4}.$$

With $|w_l| \leq 1$, each factor is at most $1/4$, giving the exponential bound in part (i). Without this weight bound, larger weights could offset small sigmoid derivatives, so small sigmoid derivatives alone do not prove that every deep network must have vanishing gradients. However, larger weights can also move the sigmoid inputs into regions where its derivative is small. Increasing the weights is therefore not a guaranteed solution.

A vector residual block adds its input back to a learned transformation of that input:

$$\mathbf{h}_{l+1} = \mathbf{h}_l + F_l(\mathbf{h}_l), \qquad F_l : \mathbb{R}^d \to \mathbb{R}^d.$$

Its Jacobian contains the partial derivative of each output component with respect to each input component. Writing the ith output component as $h_{l+1,i} = h_{l,i} + F_{l,i}(\mathbf{h}_l)$ gives

$$\begin{aligned}[J_l]_{ij} &= \frac{\partial h_{l+1,i}}{\partial h_{l,j}} \\ &= \frac{\partial h_{l,i}}{\partial h_{l,j}} + \frac{\partial F_{l,i}(\mathbf{h}_l)}{\partial h_{l,j}} \\ &= \delta_{ij} + [J_{F_l}(\mathbf{h}_l)]_{ij},\end{aligned}$$

where $\delta_{ij} = 1$ if $i = j$ and 0 otherwise, is Kroenecker's delta. These entries form the identity matrix, so

$$\boxed{J_l = I + J_{F_l}(\mathbf{h}_l)}.$$

For a scalar loss $\mathcal{L}$, the chain rule propagates its gradient backward through the block as

$$\nabla_{\mathbf{h}_l}\mathcal{L} = \big(I + J_{F_l}(\mathbf{h}_l)\big)^{\mathsf{T}}\nabla_{\mathbf{h}_{l+1}}\mathcal{L}.$$

The identity term provides a direct contribution from the next layer's gradient. When J_{F_l} is small, the block's Jacobian is close to I, which helps preserve gradients. This does not guarantee successful training: for example, $J_{F_l} = -I$ would cancel the identity term entirely, and products of Jacobians across many blocks can still shrink or amplify gradients.

B. *CNN dimensions and parameter counts.*

The input is $3 \times 24 \times 24$ in channel–height–width order. Convolutions have stride 1, no padding, dilation 1, and one bias per output channel. Pooling uses a 2×2 window with stride 2, no padding, and floor rounding.

(i) For either spatial dimension n, the output-size formula is

$$n_{\text{out}} = \left\lfloor \frac{n_{\text{in}} + 2p - d(k-1) - 1}{s} \right\rfloor + 1,$$

where k is the kernel size, p is padding, d is dilation, and s is stride. Here $p = 0$ and $d = 1$, so this becomes $n_{\text{out}} = \lfloor (n_{\text{in}} - k)/s \rfloor + 1$. Since height and width are equal, the same calculation applies to both:

Layer	Spatial calculation	Output dimension
Conv 1	$\lfloor (24-3)/1 \rfloor + 1 = 22$	$12 \times 22 \times 22$
Pool 1	$\lfloor (22-2)/2 \rfloor + 1 = 11$	$12 \times 11 \times 11$
Conv 2	$\lfloor (11-3)/1 \rfloor + 1 = 9$	$128 \times 9 \times 9$
Pool 2	$\lfloor (9-2)/2 \rfloor + 1 = 4$	$128 \times 4 \times 4$

A convolution's channel count is its number of output filters, while pooling preserves the channel count. In Pool 2, floor rounding gives 4, so the last row and column of the 9×9 input are not covered by a pooling window.

(ii) Each convolutional output filter spans all input channels. Therefore a $k \times k$ filter has $C_{\text{in}}k^2$ weights and one bias, giving

$$N_{\text{conv}} = C_{\text{out}} \cdot (C_{\text{in}}k^2 + 1).$$

For Conv 1, each output filter has $3 \cdot 3 \cdot 3 = 27$ weights and one bias. With 12 output filters, the total is

$$N_{\text{Conv 1}} = 12(27 + 1) = \boxed{336} \quad \text{(324 weights and 12 biases).}$$

Conv 2 receives 12 channels, so each output filter has $12 \cdot 3 \cdot 3 = 108$ weights and one bias. With 128 output filters,

$$N_{\text{Conv 2}} = 128(108 + 1) = \boxed{13{,}952} \quad \text{(13,824 weights and 128 biases).}$$

Max pooling only selects the maximum within each window and has no trainable weights or biases. Thus Pool 1 and Pool 2 each have $\boxed{0}$ trainable parameters.

(iii) After Pool 2, flattening the $128 \times 4 \times 4$ tensor gives

$$n_{\text{flat}} = 128 \cdot 4 \cdot 4 = \boxed{2{,}048}$$

features. A fully connected layer with 10 outputs has one weight for each input–output pair and one bias per output, so it adds

$$N_{\text{FC}} = 2{,}048 \cdot 10 + 10 = \boxed{20{,}490}$$

trainable parameters. Flattening itself has no trainable parameters.

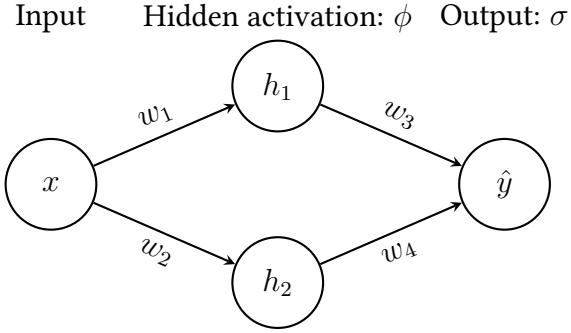
In a convolution, the same filter weights and bias are reused at every spatial location. Thus its parameter count depends on the kernel size and channel counts, rather than the number of output locations. A fully connected mapping learns separate weights for every input–output pair. For example, a fully connected mapping from the original $3 \cdot 24 \cdot 24 = 1{,}728$ inputs to the $12 \cdot 22 \cdot 22 = 5{,}808$ outputs of Conv 1 would require

$$1{,}728 \cdot 5{,}808 + 5{,}808 = 10{,}042{,}032$$

parameters, compared with Conv 1's 336. This reduction comes from both local connectivity (each output uses a 3×3 spatial patch) and weight sharing (all locations use the same filters).

2. *(Question 2.)* We are given the following network and constraints.

   For one example (x, y), where $y \in \{0, 1\}$, all four weights are scalars and there are no biases. The hidden activation ϕ changes between parts, while the output activation is always sigmoid.



The forward pass is

$$\begin{aligned} h_1 &= \phi(w_1 x), & h_2 &= \phi(w_2 x), \\ z &= w_3 h_1 + w_4 h_2, & \hat{y} &= \sigma(z), \end{aligned}$$

with binary cross-entropy loss

$$\mathcal{L} = -y \log(\hat{y}) - (1-y)\log(1-\hat{y}).$$

We use the natural logarithm for calculations.

A. Let $\phi = \sigma$. Then since $\hat{y} = \sigma(z)$, the partial derivative of the loss function $\mathcal{L}$ with respect to z is:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \hat{y}} &= -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}, \\ \frac{\partial \hat{y}}{\partial z} &= \hat{y}(1-\hat{y}), \\ \frac{\partial \mathcal{L}}{\partial z} &= \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \\ &= \left(-\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} \right) \hat{y}(1-\hat{y}) \\ &= -y(1-\hat{y}) + (1-y)\hat{y} = \boxed{\hat{y} - y}. \end{aligned}$$

We have that $z = w_3 h_1 + w_4 h_2$. Therefore:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w_3} &= \frac{\partial \mathcal{L}}{\partial z} \frac{\partial z}{\partial w_3} = (\hat{y} - y) h_1 \\ &= \boxed{\big[\sigma(w_3 \sigma(w_1 x) + w_4 \sigma(w_2 x)) - y\big] \sigma(w_1 x)}, \\ \frac{\partial \mathcal{L}}{\partial w_4} &= \frac{\partial \mathcal{L}}{\partial z} \frac{\partial z}{\partial w_4} = (\hat{y} - y) h_2 \\ &= \boxed{\big[\sigma(w_3 \sigma(w_1 x) + w_4 \sigma(w_2 x)) - y\big] \sigma(w_2 x)}. \end{aligned}$$

Let $a_1 = w_1 x$ and $a_2 = w_2 x$, with $h_1 = \sigma(a_1)$ and $h_2 = \sigma(a_2)$.

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w_1} &= \frac{\partial \mathcal{L}}{\partial z} \frac{\partial z}{\partial h_1} \frac{\partial h_1}{\partial a_1} \frac{\partial a_1}{\partial w_1} \\ &= (\hat{y} - y) \cdot w_3 \cdot h_1(1-h_1) \cdot x = (\hat{y} - y) w_3 h_1 (1-h_1) x. \end{aligned}$$

Therefore,

$$\boxed{\frac{\partial \mathcal{L}}{\partial w_1} = \big[\sigma(w_3 \sigma(w_1 x) + w_4 \sigma(w_2 x)) - y\big] w_3 \sigma(w_1 x) \big(1 - \sigma(w_1 x)\big) x}.$$

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w_2} &= \frac{\partial \mathcal{L}}{\partial z} \frac{\partial z}{\partial h_2} \frac{\partial h_2}{\partial a_2} \frac{\partial a_2}{\partial w_2} \\ &= (\hat{y} - y) \cdot w_4 \cdot h_2(1-h_2) \cdot x = (\hat{y} - y) w_4 h_2 (1-h_2) x. \end{aligned}$$

Therefore,

$$\boxed{\frac{\partial \mathcal{L}}{\partial w_2} = \big[\sigma(w_3\sigma(w_1x) + w_4\sigma(w_2x)) - y\big] w_4\sigma(w_2x)\big(1 - \sigma(w_2x)\big)x}.$$

With learning rate $\eta > 0$, the simultaneous gradient-descent updates are

$$\begin{aligned} w_1^{\text{new}} &= w_1 - \eta(\hat{y} - y)w_3h_1(1 - h_1)x, \\ w_2^{\text{new}} &= w_2 - \eta(\hat{y} - y)w_4h_2(1 - h_2)x, \\ w_3^{\text{new}} &= w_3 - \eta(\hat{y} - y)h_1, \\ w_4^{\text{new}} &= w_4 - \eta(\hat{y} - y)h_2. \end{aligned}$$

B. Let

$$\phi(u) = \frac{\log(1 + e^{2ku})}{2k}, \qquad k > 0.$$

Differentiating gives

$$\begin{aligned} \phi'(u) &= \frac{1}{2k} \cdot \frac{1}{1 + e^{2ku}} \cdot e^{2ku} \cdot 2k \\ &= \frac{e^{2ku}}{1 + e^{2ku}} = \frac{1}{1 + e^{-2ku}} = \boxed{\sigma(2ku)}. \end{aligned}$$

Thus the derivative is sigmoid evaluated at $2ku$, while the hidden outputs remain $h_1 = \phi(w_1x)$ and $h_2 = \phi(w_2x)$. The output activation and loss are unchanged, so $\partial\mathcal{L}/\partial z = \hat{y} - y$. The weight gradients are

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w_1} &= (\hat{y} - y)w_3\phi'(w_1x)x = \boxed{(\hat{y} - y)w_3\sigma(2kw_1x)x}, \\ \frac{\partial \mathcal{L}}{\partial w_2} &= (\hat{y} - y)w_4\phi'(w_2x)x = \boxed{(\hat{y} - y)w_4\sigma(2kw_2x)x}, \\ \frac{\partial \mathcal{L}}{\partial w_3} &= (\hat{y} - y)h_1 = \boxed{(\hat{y} - y)\phi(w_1x)}, \\ \frac{\partial \mathcal{L}}{\partial w_4} &= (\hat{y} - y)h_2 = \boxed{(\hat{y} - y)\phi(w_2x)}, \end{aligned}$$

where $\hat{y} = \sigma\big(w_3\phi(w_1x) + w_4\phi(w_2x)\big)$. Our chosen $\phi(u)$ approaches ReLU as k goes to infinity. For fixed $u > 0$, the limit has the indeterminate form ∞/∞. Applying L'Hôpital's rule with respect to k gives

$$\begin{aligned} \lim_{k\to\infty} \phi(u) &= \lim_{k\to\infty} \frac{\log(1 + e^{2ku})}{2k} \\ &= \lim_{k\to\infty} \frac{2ue^{2ku}/(1 + e^{2ku})}{2} \\ &= \lim_{k\to\infty} \frac{u}{1 + e^{-2ku}} = u. \end{aligned}$$

For fixed $u < 0$, $e^{2ku} \to 0$, so

$$\lim_{k\to\infty} \phi(u) = \lim_{k\to\infty} \frac{\log(1 + e^{2ku})}{2k} = 0.$$

At $u = 0$,

$$\lim_{k\to\infty} \phi(0) = \lim_{k\to\infty} \frac{\log 2}{2k} = 0.$$

Therefore,

$$\boxed{\lim_{k\to\infty} \phi(u) = \begin{cases} u, & u > 0, \\ 0, & u \leq 0 \end{cases} = \max(0, u) = \mathrm{ReLU}(u)}.$$

ReLU is differentiable for $u \neq 0$, with derivative 0 for $u < 0$ and 1 for $u > 0$. At $u = 0$, its ordinary derivative does not exist, but we can choose a value for the purposes of backpropogation. Since $u = 0$ is the boundary between a zero and non-zero derivative, a choice of 0 for the derivative makes sense, but we could choose another value between 0 and 1 instead. This supplies a value for the chain-rule factor when a unit's input is exactly zero, allowing backpropagation to proceed. Choosing 1 is also valid, but does not change the zero derivative for negative inputs. Thus the smooth activation ϕ is an alternative to ReLU, rather than a prerequisite for training with ReLU.

C. We set

$$x = 0.7, \qquad y = 1, \qquad (w_1, w_2, w_3, w_4) = (0.2, -0.3, 0.4, -0.5).$$

For sigmoid and scaled Softplus with $k = 1$, and compare the analytic gradients with automatic differentiation and central finite differences using $\varepsilon = 10^{-5}$ and float64 arithmetic. The four gradients and the maximum absolute discrepancy for each comparison are reported below. The code is in `hw3_question2c-tbelcher.ipynb` and uses NumPy 1.26.4 and Autograd 1.9.1. The loss is evaluated directly from logits as $\log(1 + e^z) - yz$ using `logaddexp` for numerical stability.

Sigmoid hidden activation

Weight	Analytic	Autodiff	Central difference
w_1	-0.035000918963	-0.035000918963	-0.035000918963
w_2	0.043484696482	0.043484696482	0.043484696477
w_3	-0.268791282082	-0.268791282082	-0.268791282082
w_4	-0.224950589689	-0.224950589689	-0.224950589683

Scaled Softplus hidden activation ($k = 1$)

Weight	Analytic	Autodiff	Central difference
w_1	-0.078049347229	-0.078049347229	-0.078049347224
w_2	0.067922216435	0.067922216435	0.067922216440
w_3	-0.206270074531	-0.206270074531	-0.206270074532
w_4	-0.123587760488	-0.123587760488	-0.123587760492

The maximum absolute discrepancies across the four weights, computed before rounding the table entries, are

Activation	Analytic vs. autodiff	Analytic vs. central difference
Sigmoid	5.551115×10^{-17}	5.799611×10^{-12}
Scaled Softplus	0	5.580245×10^{-12}

Both comparisons agree closely with the analytic gradients. For each weight w_i, central finite differences compute the average rate of change between $w_i - \varepsilon$ and $w_i + \varepsilon$, holding all other weights fixed:

$$\frac{\partial \mathcal{L}}{\partial w_i} \approx \frac{\mathcal{L}(w_i + \varepsilon) - \mathcal{L}(w_i - \varepsilon)}{2\varepsilon}.$$

Here $\mathcal{L}(w_i \pm \varepsilon)$ denotes the loss after changing only w_i. A smaller ε makes this average rate of change closer to the derivative in exact arithmetic. However, float64 stores each computed loss with finite precision. As ε decreases, the two loss values become nearly equal, and their difference can become comparable to their rounding errors. Dividing that difference by the small number 2ε amplifies those errors. Thus reducing ε can initially improve agreement, but an excessively small step can worsen it.